VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Microproccesser - Microcontroller

(CO3009)

LAB 3: BUTTONS and SWITCHES

Tutors: TS Lê Trọng Nhân
Th.S Phan Văn Sỹ

Student: Trương Thiên Ân
Student ID: 2310190
Class: TN01

Table of contents

1 Objectives

In this lab, you will

- Learn how to add new C source files and C header files in an STM32 project,
- Learn how to read digital inputs and display values to LEDs using a timer interrupt of a microcontroller (MCU).
- Learn how to debounce when reading a button.
- Learn how to create an FSM and implement an FSM in an MCU.

2 Introduction

Embedded systems usually use buttons (or keys, or switches, or any form of mechanical contacts) as part of their user interface. This general rule applies from the most basic remote-control system for opening a garage door, right up to the most sophisticated aircraft autopilot system. Whatever the system you create, you need to be able to create a reliable button interface.
A button is generally hooked up to an MCU so as to generate a certain logic level when pushed or closed or “active" and the opposite logic level when unpushed or open or “inactive." The active logic level can be either ‘0’ or ‘1’, but for reasons both historical and electrical, an active level of ’0’ is more common.
We can use a button if we want to perform operations such as:

- Drive a motor while a switch is pressed.
- Switch on a light while a switch is pressed.
- Activate a pump while a switch is pressed.

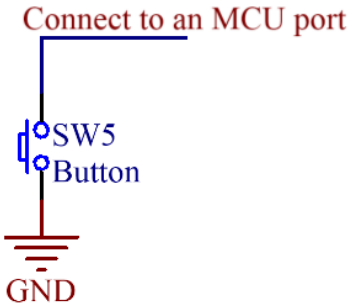
These operations could be implemented using an electrical button without using an MCU; however, use of an MCU may well be appropriate if we require more complex behaviours. For example:

- Drive a motor while a switch is pressed.
  Condition: If the safety guard is not in place, don’t turn the motor. Instead sound a buzzer for 2 seconds.
- Switch on a light while a switch is pressed.
  Condition: To save power, ignore requests to turn on the light during daylight hours.
- Activate a pump while a switch is pressed
  Condition: If the main water reservoir is below 300 litres, do not start the main pump: instead, start the reserve pump and draw the water from the emergency tank.

In this lab, we consider how you read inputs from mechanical buttons in your embedded application using an MCU.

3 Basic techniques for reading from port pins

3.1 The need for pull-up resistors

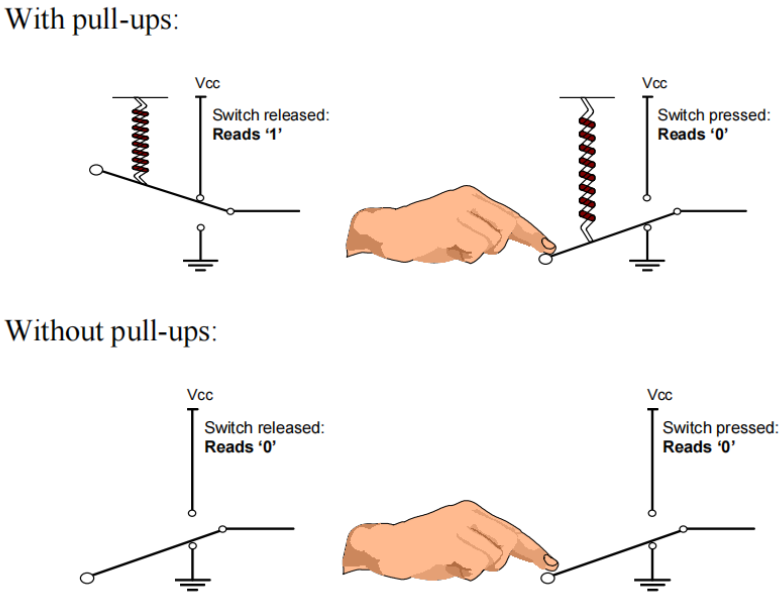


Hình 1: *Connecting a button to an MCU*

Figure 1 shows a way to connect a button to an MCU. This hardware operates as follows:

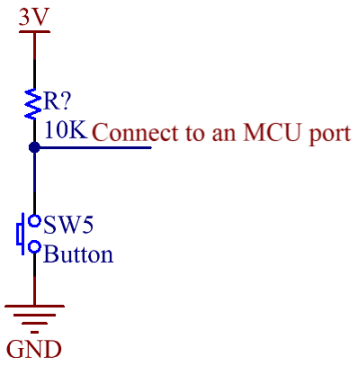
- When the switch is open, it has no impact on the port pin. An internal resistor on the port “pulls up" the pin to the supply voltage of the MCU (typically 3.3V for STM32F103). If we read the pin, we will see the value ‘1’.
- When the switch is closed (pressed), the pin voltage will be 0V. If we read the pin, we will see the value ‘0’.

However, if the MCU does not have a pull-up resistor inside, when the button is pressed, the read value will be ‘0’, but even we release the button, the read value is still ‘0’ as shown in Figure 2.



Hình 2: *The need of pull up resistors*

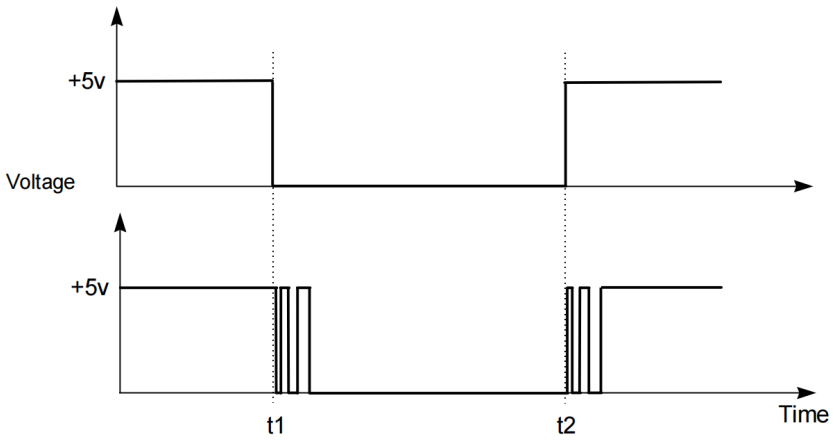
So a reliable way to connect a button/switch to an MCU is that we explicitly use an external pull-up resistor as shown in Figure 3.



Hình 3: *A reliable way to connect a button to an MCU*

3.2 Dealing with switch bounces

In practice, all mechanical switch contacts bounce (that is, turn on and off, repeatedly, for short period of time) after the switch is closed or opened as shown in Figure 4.



Hình 4: *Switch bounces*

Every system that uses any kind of mechanical switch must deal with the issue of debouncing. The key task is to make sure that one mechanical switch or button action is only read as one action by the MCU, even though the MCU will typically be fast enough to detect the unwanted switch bounces and treat them as separate events. Bouncing can be eliminated by special ICs or by RC circuitry, but in most cases debouncing is done in software because software is “free".

As far as the MCU concerns, each “bounce" is equivalent to one press and release of an “ideal” switch. Without appropriate software design, this can give several problems:

- Rather than reading ‘A’ from a keypad, we may read ‘AAAAA’
- Counting the number of times that a switch is pressed becomes extremely difficult
- If a switch is depressed once, and then released some time later, the ‘bounce’ may make it appear as if the switch has been pressed again (at the time of release).

The key to debouncing is to establish a minimum criterion for a valid button push, one that can be implemented in software. This criterion must involve differences in time - two button presses in 20ms must be treated as one button event, while two button presses in 2 seconds must be treated as two button events. So what are the relevant times we need to consider? They are these:

- Bounce time: most buttons seem to stop bouncing within 10ms
- Button press time: the shortest time a user can press and release a button seems to be between 50 and 100ms
- Response time: a user notices if the system response is 100ms after the button press, but not if it is 50ms after

Combining all of these times, we can set a few goals

- Ignore all bouncing within 10ms
- Provide a response within 50ms of detecting a button push (or release)
- Be able to detect a 50ms push and a 50ms release

The simplest debouncing method is to examine the keys (or buttons or switches) every N milliseconds, where N > 10ms (our specified button bounce upper limit) and N <= 50ms (our specified response time). We then have three possible outcomes every time we read a button:

- We read the button in the solid '0' state
- We read the button in the solid '1' state
- We read the button while it is bouncing (so we will get either a '0' or a '1')

Outcomes 1 and 2 pose no problems, as they are what we would always like to happen. Outcome 3 also poses no problem because during a bounce either state is acceptable. If we have just pressed an active-low button and we read a '1' as it bounces, the next time through we are guaranteed to read a '0' (remember, the next time through all bouncing will have ceased), so we will just detect the button push a bit later. Otherwise, if we read a '0' as the button bounces, it will still be '0' the next time after all bouncing has stopped, so we are just detecting the button push a bit earlier. The same applies to releasing a button. Reading a single bounce (with all bouncing over by the time of the next read) will never give us an invalid button state. It's only reading multiple bounces (multiple reads while bouncing is occurring) that can give invalid button states such as repeated push signals from one physical push.

So if we guarantee that all bouncing is done by the time we next read the button, we're good. Well, almost good, if we're lucky...

MCUs often live among high-energy beasts, and often control the beasts. High energy devices make electrical noise, sometimes great amounts of electrical noise. This noise can, at the worst possible moment, get into your delicate button-and-high-value-pullup circuit and act like a real button push. Oops, missile launched, sorry!

If the noise is too intense we cannot filter it out using only software, but will need hardware of some sort (or even a redesign). But if the noise is only occasional, we can filter it out in software without too much bother. The trick is that instead of regarding a single button 'make' or 'break' as valid, we insist on N contiguous makes or breaks to mark a valid button event. N will be a factor of your button scanning rate and the amount of filtering you want to add. Bigger N gives more filtering. The simplest filter (but still a big improvement over no filtering) is just an N of 2, which means compare the current button state with the last button state, and only if both are the same is the output valid.

Note that now we have not two but three button states: active (or pressed), inactive (or released), and indeterminate or invalid (in the middle of filtering, not yet filtered). In most cases we can treat the invalid state the same as the inactive state, since we care in most cases only about when we go active (from whatever state) and when we cease being active (to inactive or invalid). With that simplification we can look at simple N = 2 filtering reading a button wired to STM32 MCU:

```
1 void button_reading(void){
```

```
    static unsigned char last_button;
    unsigned char raw_button;
    unsigned char filtered_button;
    last_button = raw_button;
    raw_button = HAL_GPIO_ReadPin(BUTTON_1_GPIO_Port,
  BUTTON_1_Pin);
    if(last_button == raw_button){
        filtered_button = raw_button;
    }
}
```

Program 1: Read port pin and deboucing

The function button_reading() must be called no more often than our debounce time (10ms).
To expand to greater filtering (larger N), keep in mind that the filtering technique essentially involves reading the current button state and then either counting or reseting the counter. We count if the current button state is the same as the last button state, and if our count reaches N we then report a valid new button state. We reset the counter if the current button state is different than the last button state, and we then save the current button state as the new button state to compare against the next time. Also note that the larger our value of N the more often our filtering routine must be called, so that we get a filtered response within our specified 50ms deadline. So for example with an N of 8 we should be calling our filtering routine every 2 - 5ms, giving a response time of 16 - 40ms (>10ms and <50ms).

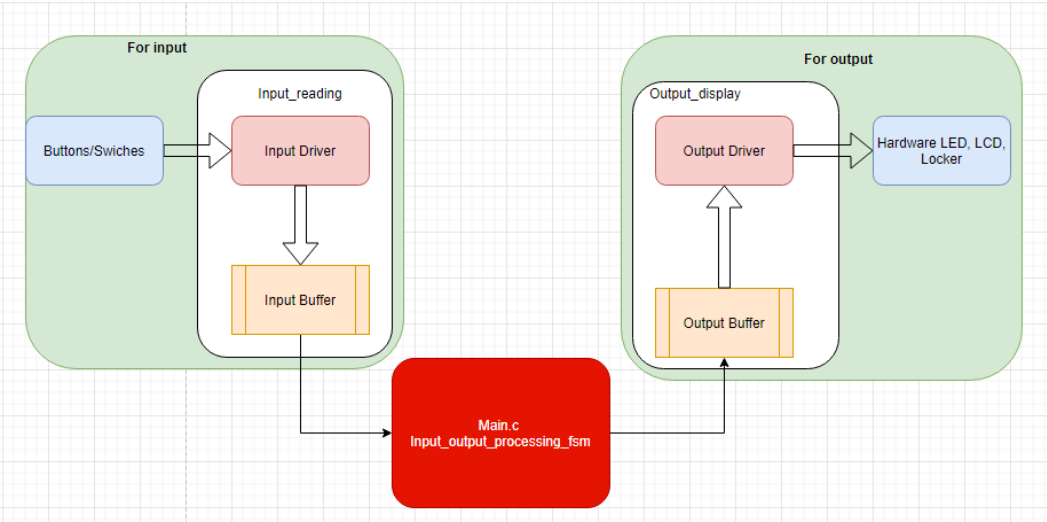
4 Reading switch input (basic code) using STM32

To demonstrate the use of buttons/switches in STM32, we use an example which requires to write a program that

- Has a timer which has an interrupt in every 10 milliseconds.
- Reads values of button PB0 every 10 milliseconds.
- Increases the value of LEDs connected to PORTA by one unit when the button PB0 is pressed.
- Increases the value of PORTA automatically in every 0.5 second, if the button PB0 is pressed in more than 1 second.

4.1 Input Output Processing Patterns

For both input and output processing, we have a similar pattern to work with. Normally, we have a module named driver which works directly to the hardware. We also have a buffer to store temporarily values. In the case of input processing, the driver will store the value of the hardware status to the buffer for further processing. In the case of output processing, the driver uses the buffer data to output to the hardware.



Hình 5: *Input Output Processing Patterns*

Figure 5 shows that we should have an *input_ reading* module to processing the buttons, then store the processed data to the buffer. Then a module of *input_ output_ processing_ fsm* will process the input data, and update the output buffer. The output driver gets the value from the output buffer to transfer to the hardware.

4.2 Setting up

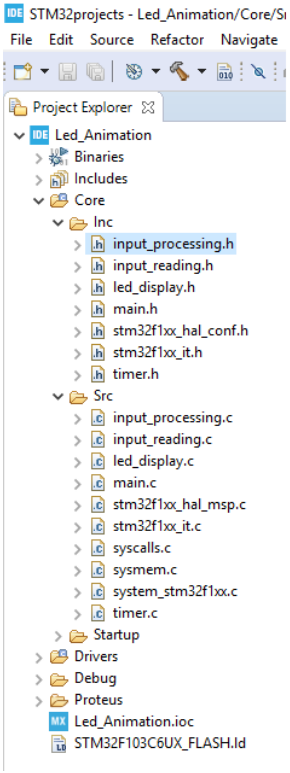
4.2.1 Create a project

Please follow the instruction in Labs 1 and 2 to create a project that includes:

- PB0 as an input port pin,
- PA0-PA7 as output port pins, and
- Timer 2 10ms interrupt

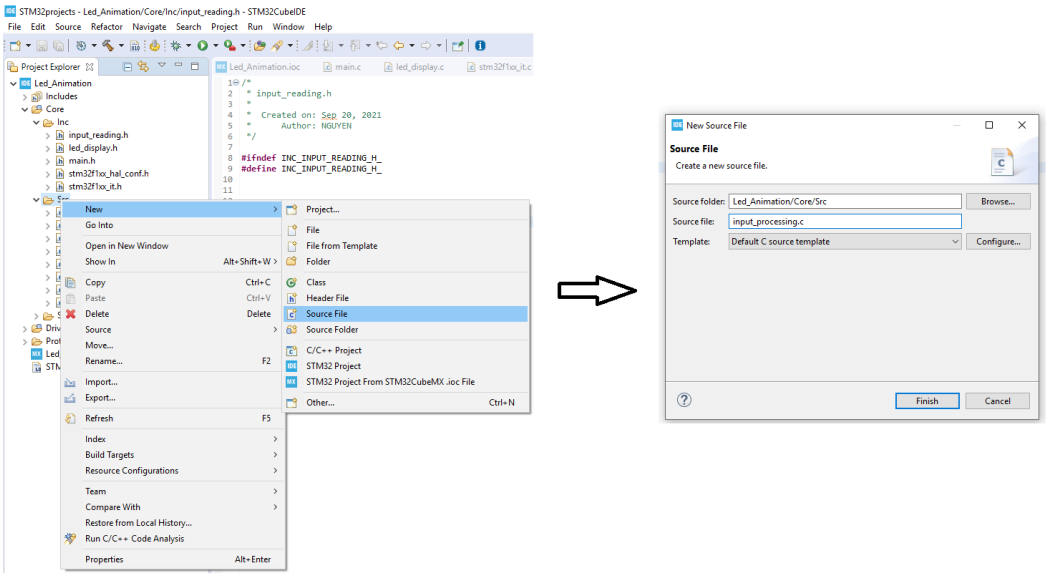
4.2.2 Create a file C source file and header file for input reading

We are expected to have files for button processing and led display as shown in Figure 6.

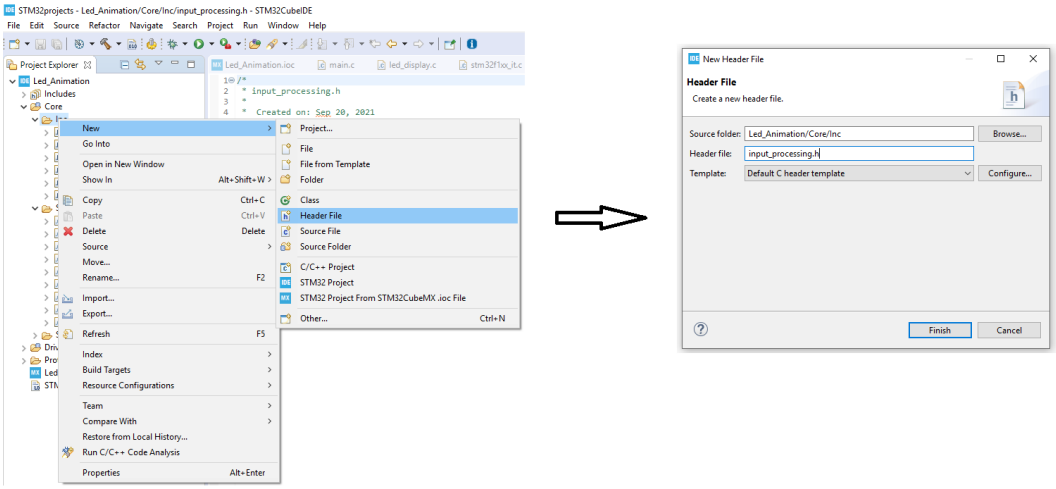


Hình 6: *File Organization*

Steps 1 (Figure 7): Right click to the folder **Src**, select **New**, then select **Source File**. There will be a pop-up. Please type the file name, then click **Finish**.
Step 2 (Figure 8): Do the same for the C header file in the folder **Inc**.



Hình 7: *Step 1: Create a C source file for input reading*



Hình 8: *Step 2: Create a C header file for input processing*

4.3 Code For Read Port Pin and Debouncing

4.3.1 The code in the input_reading.c file

```
#include "main.h"
//we aim to work with more than one buttons
#define NO_OF_BUTTONS                 1
//timer interrupt duration is 10ms, so to pass 1 second,
//we need to jump to the interrupt service routine 100 time
#define DURATION_FOR_AUTO_INCREASING     100
#define BUTTON_IS_PRESSED                  GPIO_PIN_RESET
#define BUTTON_IS_RELEASED                 GPIO_PIN_SET
//the buffer that the final result is stored after
//debouncing
static GPIO_PinState buttonBuffer[NO_OF_BUTTONS];
//we define two buffers for debouncing
static GPIO_PinState debounceButtonBuffer1[NO_OF_BUTTONS];
static GPIO_PinState debounceButtonBuffer2[NO_OF_BUTTONS];
//we define a flag for a button pressed more than 1 second.
static uint8_t flagForButtonPress1s[NO_OF_BUTTONS];
//we define counter for automatically increasing the value
//after the button is pressed more than 1 second.
static uint16_t counterForButtonPress1s[NO_OF_BUTTONS];
void button_reading(void){
  for(char i = 0; i < NO_OF_BUTTONS; i ++){
    debounceButtonBuffer2[i] =debounceButtonBuffer1[i];
    debounceButtonBuffer1[i] = HAL_GPIO_ReadPin(
   BUTTON_1_GPIO_Port, BUTTON_1_Pin);
    if(debounceButtonBuffer1[i] == debounceButtonBuffer2[i
   ])
       buttonBuffer[i] = debounceButtonBuffer1[i];
       if(buttonBuffer[i] == BUTTON_IS_PRESSED){
       //if a button is pressed, we start counting
         if(counterForButtonPress1s[i] <
   DURATION_FOR_AUTO_INCREASING){
           counterForButtonPress1s[i]++;
         } else {
         //the flag is turned on when 1 second has passed
         //since the button is pressed.
           flagForButtonPress1s[i] = 1;
           //todo
         }
       } else {
         counterForButtonPress1s[i] = 0;
         flagForButtonPress1s[i] = 0;
       }
  }
}
```

Program 2: Define constants buffers and button_reading function

```c
unsigned char is_button_pressed(uint8_t index){
  if(index >= NO_OF_BUTTONS) return 0;
  return (buttonBuffer[index] == BUTTON_IS_PRESSED);
}
```

Program 3: Checking a button is pressed or not

```c
unsigned char is_button_pressed_1s(unsigned char index){
  if(index >= NO_OF_BUTTONS) return 0xff;
  return (flagForButtonPress1s[index] == 1);
}
```

Program 4: Checking a button is pressed more than a second or not

4.3.2 The code in the input_reading.h file

```c
#ifndef INC_INPUT_READING_H_
#define INC_INPUT_READING_H_
void button_reading(void);
unsigned char is_button_pressed(unsigned char index);
unsigned char is_button_pressed_1s(unsigned char index);
#endif /* INC_INPUT_READING_H_ */
```

Program 5: Prototype in input_reading.h file

4.3.3 The code in the timer.c file

```c
#include "main.h"
#include "input_reading.h"

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if(htim->Instance == TIM2){
    button_reading();
  }
}
```

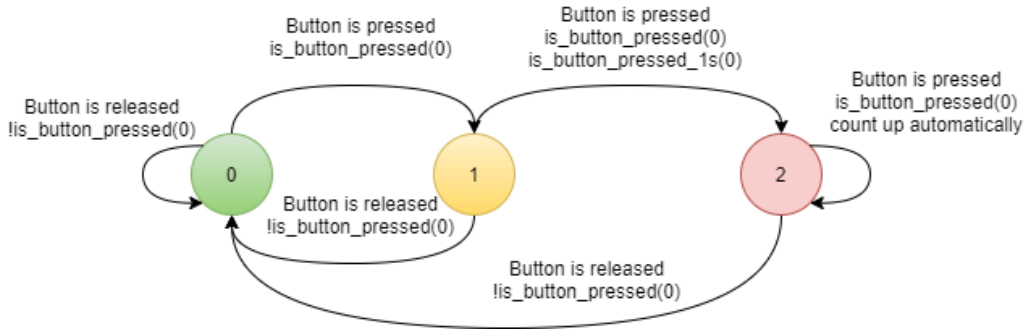
Program 6: Timer interrupt callback function

4.4 Button State Processing

4.4.1 Finite State Machine

To solve the example problem, we define 3 states as follows:

- State 0: The button is released or the button is in the initial state.
- State 1: When the button is pressed, the FSM will change to State 1 that is increasing the values of PORTA by one value. If the button is released, the FSM goes back to State 0.
- State 2: while the FSM is in State 1, the button is kept pressing more than 1 second, the state of FSM will change from 1 to 2. In this state, if the button is kept pressing, the value of PORTA will be increased automatically in every 500ms. If the button is released, the FSM goes back to State 0.



Hình 9: *An FSM for processing a button*

4.4.2 The code for the FSM in the input_processing.c file

Please note that *fsm_for_input_processing* function should be called inside the super loop of the main functin.

```
#include "main.h"
#include "input_reading.h"

enum ButtonState{BUTTON_RELEASED, BUTTON_PRESSED,
   BUTTON_PRESSED_MORE_THAN_1_SECOND} ;
enum ButtonState buttonState = BUTTON_RELEASED;
void fsm_for_input_processing(void){
  switch(buttonState){
  case BUTTON_RELEASED:
    if(is_button_pressed(0)){
      buttonState = BUTTON_PRESSED;
      //INCREASE VALUE OF PORT A BY ONE UNIT
    }
    break;
  case BUTTON_PRESSED:
    if(!is_button_pressed(0)){
      buttonState = BUTTON_RELEASED;
    } else {
      if(is_button_pressed_1s(0)){
        buttonState = BUTTON_PRESSED_MORE_THAN_1_SECOND;
      }
    }
    break;
  case BUTTON_PRESSED_MORE_THAN_1_SECOND:
    if(!is_button_pressed(0)){
      buttonState = BUTTON_RELEASED;
    }
    //todo
    break;
  }
}
```

Program 7: The code in the input_processing.c file

4.4.3 The code in the input_processing.h

```
#ifndef INC_INPUT_PROCESSING_H_
#define INC_INPUT_PROCESSING_H_

void fsm_for_input_processing(void);

#endif /* INC_INPUT_PROCESSING_H_ */
```

Program 8: Code in the input_processing.h file

4.4.4 The code in the main.c file

```c
#include "main.h"
#include "input_processing.h"
//don't modify this part
int main(void){
    HAL_Init();
    /* Configure the system clock */
    SystemClock_Config();
    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_TIM2_Init();
    while (1)
    {
        //you only need to add the fsm function here
        fsm_for_input_processing();
    }
}
```

Program 9: The code in the main.c file

5 Exercises and Report

5.1 Specifications

You are required to build an application of a traffic light in a cross road which includes some features as described below:

- The application has 12 LEDs including 4 red LEDs, 4 amber LEDs, 4 green LEDs.
- The application has 4 seven segment LEDs to display time with 2 for each road. The 2 seven segment LEDs will show time for each color LED corresponding to each road.
- The application has three buttons which are used
  - to select modes,
  - to modify the time for each color led on the fly, and
  - to set the chosen value.
- The application has at least 4 modes which is controlled by the first button. Mode 1 is a normal mode, while modes 2 3 4 are modification modes. You can press the first button to change the mode. Modes will change from 1 to 4 and back to 1 again.

  Mode 1 - Normal mode:
  - The traffic light application is running normally.

  Mode 2 - Modify time duration for the red LEDs: This mode allows you to change the time duration of the red LED in the main road. The expected behaviours of this mode include:
  - All single red LEDs are blinking in 2 Hz.
  - Use two seven-segment LEDs to display the value.
  - Use the other two seven-segment LEDs to display the mode.
  - The second button is used to increase the time duration value for the red LEDs.
  - The value of time duration is in a range of 1 - 99.
  - The third button is used to set the value.

  Mode 3 - Modify time duration for the amber LEDs: Similar for the red LEDs described above with the amber LEDs.

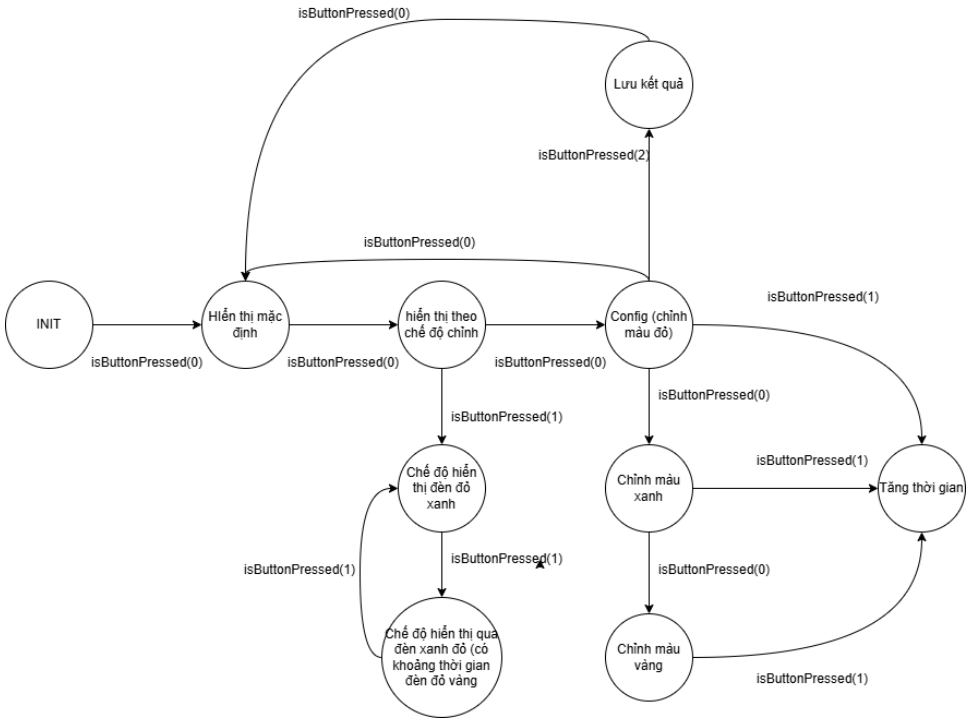
  Mode 4 - Modify time duration for the green LEDs: Similar for the red LEDs described above with the green LEDs.

5.2 Exercise 1: Sketch an FSM

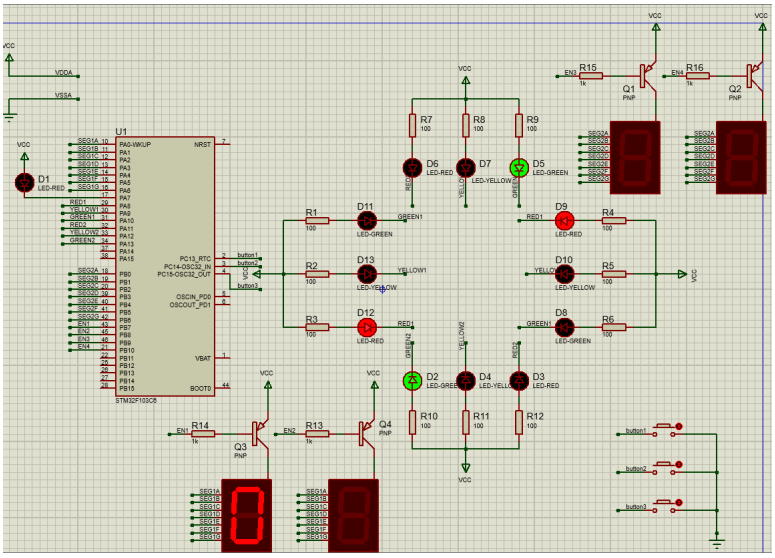
Your task in this exercise is to sketch an FSM that describes your idea of how to solve the problem.
Please add your report here.

5.3 Exercise 2: Proteus Schematic

Your task in this exercise is to draw a Proteus schematic for the problem above.



Hình 10: *Finate state machine of the project*



Hình 11: *Schematic of the project*

Exercise 3 to exercise 10 is reported below.

5.4 Exercise 10: To finish the project

Your tasks in this exercise are:

- To integrate all the previous tasks to one final project
- To create a video to show all features in the specification
- To add a report to describe your solution for each exercise.
- To submit your report and code on the BKeL

This is my source code with full explainations

5.4.1 global.c / global.h

These files are used to define all constants, varriable of my project. **global.h**

```
#ifndef INC_GLOBAL_H_
#define INC_GLOBAL_H_

#include "7seg.h"
#include "state_machine.h"
#include "led_display.h"
#include "software_timer.h"
#include "button.h"
#include "main.h"

//main state
#define INIT       0
#define AUTO_STATE    1
#define MANUAL_STATE  2
#define CONFIG_STATE  3
extern int mode;

//light state

#define NORMAL_RED    11
#define NORMAL_YELLOW 12
#define NORMAL_GREEN  13

#define BLINK_RED   21
#define BLINK_YELLOW  22
#define BLINK_GREEN   23
#define BLINK_RED_GREEN   24
#define BLINK_RED_YELLOW  25
#define BLINK_GREEN_RED   26
extern int manual_blink_mode;

//config
#define CONFIG_RED    31
#define CONFIG_GREEN  32
#define CONFIG_YELLOW 33
extern int config_mode;

//button
#define LONG_PRESSED_TIME  200 //(200ms)
#define NORMAL_STATE  GPIO_PIN_SET
#define PRESSED_STATE GPIO_PIN_RESET
#define MAX_BUTTON    3
extern int longPressedTime[MAX_BUTTON];
extern int button_flag[MAX_BUTTON];
extern int button_long_flag[MAX_BUTTON];
extern int keyReg0[MAX_BUTTON];
```

```
47 extern int keyReg1[MAX_BUTTON];
48 extern int keyReg2[MAX_BUTTON];
49 extern int keyReg3[MAX_BUTTON];
50
51 //Timer
52 #define MAX_TIMER 20
53 extern int timer_flag[MAX_TIMER];
54 extern int timer_counter[MAX_TIMER];
55
56 //LED and 7SEG
57 extern uint8_t led7seg_num_display[];
58 extern int led_buffer[4];
59 extern int led_index;
60 extern int red_time, green_time, yellow_time;
61 extern int road1_state, road2_state;
62 extern int road1_counter, road2_counter;
63 extern int counter;
64 #endif /* INC_GLOBAL_H_ */
```

Program 10: Define constants of all variables

global.c

```
1  #include "global.h"
2  int mode = INIT;
3  int red_time = 9, green_time = 6, yellow_time = 3;
4  int road1_state = NORMAL_RED, road2_state = NORMAL_GREEN;
5  int road1_counter, road2_counter;
6  int manual_blink_mode = INIT;
7  int config_mode = CONFIG_RED;
8  int counter = 0;
9  int led_buffer[4] = {0};
10 uint8_t led7seg_num_display[] = {
11     /*0*/ 0x40,/*0b1000000*/
12     /*1*/ 0x79,/*0b11111001*/
13     /*2*/ 0x24,/*0b0100100*/
14     /*3*/ 0x30,/*0b0110000*/
15     /*4*/ 0x19,/*0b0011001*/
16     /*5*/ 0x12,/*0b0010010*/
17     /*6*/ 0x02,/*0b0000010*/
18     /*7*/ 0x78,/*0b1111000*/
19     /*8*/ 0x00,/*0b0000000*/
20     /*9*/ 0x10/*0b0010000*/
21 };
22 int led_index = 0;
23 int keyReg0[MAX_BUTTON];
24 int keyReg1[MAX_BUTTON];
25 int keyReg2[MAX_BUTTON];
26 int keyReg3[MAX_BUTTON];
27 int longPressedTime[MAX_BUTTON];
28 int button_flag[MAX_BUTTON];
```

```
int button_long_flag[MAX_BUTTON];
```

Program 11: global.c

5.4.2 software_timer.c/ software_time.h

These header and source code files are in charge of managing all processes in this project

```
#ifndef INC_SOFTWARE_TIMER_H_
#define INC_SOFTWARE_TIMER_H_
#include "global.h"

void setTimer(int timer_id, int dur);
int isExpired(int timer_id);
void timerRun();

#endif /* INC_SOFTWARE_TIMER_H_ */
```

Program 12: software_timer.h

```
#include "global.h"
int timer_flag[MAX_TIMER];
int timer_counter[MAX_TIMER];

void setTimer(int timer_id, int dur){
  if (timer_id < 0 || timer_id >= MAX_TIMER){
    return;
  }
  timer_counter[timer_id] = dur;
  timer_flag[timer_id] = 0;
}

int isExpired(int timer_id){
  if (timer_flag[timer_id] == 1){
    timer_flag[timer_id] = 0;
    return 1;
  }
  return 0;
}

void timerRun(){
  for (int i = 0; i < MAX_TIMER; i++){
    if (timer_counter[i] > 0){
      timer_counter[i]--;
    }
    if (timer_counter[i] == 0){
      timer_flag[i] = 1;
    }
  }
```

```
}
```

Program 13: software_timer.h

5.4.3 button.h/button.c

These 2 files are reponsible for receiving and processing buttons input.

```
#ifndef INC_BUTTON_H_
#define INC_BUTTON_H_

#include "global.h"

void subKeyProcess(int index);
void getKeyInput();
int isButtonPressed(int index);
int isButtonLongPressed(int index);


#endif /* INC_BUTTON_H_ */
```

Program 14: button.h

```
#include "button.h"

#define LONG_PRESSED_TIME  200 //(200ms)

GPIO_TypeDef* BUTTON_PORT[MAX_BUTTON] = {BUTTON1_GPIO_Port,
    BUTTON2_GPIO_Port, BUTTON3_GPIO_Port};
uint16_t BUTTON_PIN[MAX_BUTTON] = {BUTTON1_Pin, BUTTON2_Pin
   , BUTTON3_Pin};


void subKeyProcess(int index){
  button_flag[index] = 1;
}

int isButtonPressed(int index){
  if (button_flag[index] == 1){
    button_flag[index] = 0;
    return 1;
  }else{
    return 0;
  }
}

int isButtonLongPressed(int index){
  if (button_long_flag[index] == 1){
    button_long_flag[index] = 0;
    return 1;
  }else{
```

```
    return 0;
  }
}

void getKeyInput(){
  for (int i = 0; i < MAX_BUTTON; i++){
    keyReg2[i] = keyReg1[i];
    keyReg1[i] = keyReg0[i];
    keyReg0[i] = HAL_GPIO_ReadPin(BUTTON_PORT[i],
   BUTTON_PIN[i]);
    if ((keyReg0[i] == keyReg1[i]) && (keyReg1[i] ==
   keyReg2[i])){
      if (keyReg3[i] != keyReg2[i]){
        keyReg3[i] = keyReg2[i];
        if(keyReg3[i] == PRESSED_STATE){
          subKeyProcess(i);
          longPressedTime[i] = LONG_PRESSED_TIME; //(ms)
        }
      }else{
        if(keyReg3[i] == PRESSED_STATE){
          if (longPressedTime[i] > 0){
            longPressedTime[i]--;
            if(longPressedTime[i] == 0){
              longPressedTime[i] = LONG_PRESSED_TIME;
              button_long_flag[i] = 1;
            }
          }
        }
      }
    }
  }
}
```

Program 15: button.c

5.4.4 7seg.c/ 7seg.h

the main purpose of these files is to control 7-segment LED and display 7-segment LED

```
#ifndef INC_7SEG_H_
#define INC_7SEG_H_

#include "global.h"

void display7SEG(int road_way, int num);
void update7SEG(int index);
void updateLEDBuffer(int road_way, int num);
void clear7SEG();
//note that there is 4 7SEG: index 0 and 1 is for way 1, 2
   and 3 for way 2
```

```
#endif /* INC_7SEG_H_ */
```

Program 16: 7seg.h

```
#include "global.h"
void display7SEG(int road_way, int num){
  if (road_way != 1 && road_way !=2){
    return;
  }
  if (num < 0 || num > 9){
    return;
  }
  switch (road_way){
    case 1:
      HAL_GPIO_WritePin(SEG1_1_GPIO_Port, SEG1_1_Pin, !((~led7seg_num_display[num]) & (1 << 0)));
      HAL_GPIO_WritePin(SEG1_2_GPIO_Port, SEG1_2_Pin, !((~led7seg_num_display[num]) & (1 << 1)));
      HAL_GPIO_WritePin(SEG1_3_GPIO_Port, SEG1_3_Pin, !((~led7seg_num_display[num]) & (1 << 2)));
      HAL_GPIO_WritePin(SEG1_4_GPIO_Port, SEG1_4_Pin, !((~led7seg_num_display[num]) & (1 << 3)));
      HAL_GPIO_WritePin(SEG1_5_GPIO_Port, SEG1_5_Pin, !((~led7seg_num_display[num]) & (1 << 4)));
      HAL_GPIO_WritePin(SEG1_6_GPIO_Port, SEG1_6_Pin, !((~led7seg_num_display[num]) & (1 << 5)));
      HAL_GPIO_WritePin(SEG1_7_GPIO_Port, SEG1_7_Pin, !((~led7seg_num_display[num]) & (1 << 6)));
      break;
    case 2:
      HAL_GPIO_WritePin(SEG2_1_GPIO_Port, SEG2_1_Pin, !((~led7seg_num_display[num]) & (1 << 0)));
      HAL_GPIO_WritePin(SEG2_2_GPIO_Port, SEG2_2_Pin, !((~led7seg_num_display[num]) & (1 << 1)));
      HAL_GPIO_WritePin(SEG2_3_GPIO_Port, SEG2_3_Pin, !((~led7seg_num_display[num]) & (1 << 2)));
      HAL_GPIO_WritePin(SEG2_4_GPIO_Port, SEG2_4_Pin, !((~led7seg_num_display[num]) & (1 << 3)));
      HAL_GPIO_WritePin(SEG2_5_GPIO_Port, SEG2_5_Pin, !((~led7seg_num_display[num]) & (1 << 4)));
      HAL_GPIO_WritePin(SEG2_6_GPIO_Port, SEG2_6_Pin, !((~led7seg_num_display[num]) & (1 << 5)));
      HAL_GPIO_WritePin(SEG2_7_GPIO_Port, SEG2_7_Pin, !((~led7seg_num_display[num]) & (1 << 6)));
      break;
    default:
      break;
  }
}

```

```
33 void update7SEG(int index){
34   HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
35   HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_SET);
36   HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_SET);
37   HAL_GPIO_WritePin(EN4_GPIO_Port, EN4_Pin, GPIO_PIN_SET);
38   switch(index){
39     case 0:
40       display7SEG(1, led_buffer[0]);
41       HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin,
     GPIO_PIN_RESET);
42       break;
43     case 1:
44       display7SEG(1, led_buffer[1]);
45 //       HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin,
     GPIO_PIN_SET);
46       HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin,
     GPIO_PIN_RESET);
47       break;
48     case 2:
49       display7SEG(2, led_buffer[2]);
50       HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin,
     GPIO_PIN_RESET);
51 //       HAL_GPIO_WritePin(EN4_GPIO_Port, EN4_Pin,
     GPIO_PIN_SET);
52       break;
53     case 3:
54       display7SEG(2, led_buffer[3]);
55 //       HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin,
     GPIO_PIN_SET);
56       HAL_GPIO_WritePin(EN4_GPIO_Port, EN4_Pin,
     GPIO_PIN_RESET);
57       break;
58     default:
59       break;
60   }
61 }
62
63 void updateLEDBuffer(int road_way, int num){
64   switch (road_way){
65     case 1:
66       led_buffer[0] = num / 10;
67       led_buffer[1] = num % 10;
68       break;
69     case 2:
70       led_buffer[2] = num / 10;
71       led_buffer[3] = num % 10;
72       break;
73     default:
74       break;
```

```
  }
}
```

Program 17: 7seg.c

5.4.5 display_led.h/ display_led.c

This code is used to display the traffic system in different modes (12 LEDs, 4 7-segment LEDs)

```
#ifndef INC_LED_DISPLAY_H_
#define INC_LED_DISPLAY_H_

#include "global.h"

void updateTrafficLight(int state);
void displayAutoTrafficLight();
void displayBlinkTrafficLight();
void displayTime();
void updateCounter();
void clearLED();

#endif /* INC_LED_DISPLAY_H_ */
```

Program 18: display_led.h

```
    #include "global.h"
int one_sec_tick = 0;

void updateTrafficLight(int state) {
  if (mode == AUTO_STATE){
    switch (road1_state){
      case NORMAL_RED:
        HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin,
    GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port,
   LED_GREEN1_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(LED_YELLOW1_GPIO_Port,
   LED_YELLOW1_Pin, GPIO_PIN_SET);
        break;
      case NORMAL_GREEN:
        HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin,
    GPIO_PIN_SET);
        HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port,
   LED_GREEN1_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED_YELLOW1_GPIO_Port,
   LED_YELLOW1_Pin, GPIO_PIN_SET);
        break;
      case NORMAL_YELLOW:
        HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin,
    GPIO_PIN_SET);
```

```
19          HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port,
    LED_GREEN1_Pin, GPIO_PIN_SET);
20          HAL_GPIO_WritePin(LED_YELLOW1_GPIO_Port,
    LED_YELLOW1_Pin, GPIO_PIN_RESET);
21          break;
22        default:
23          break;
24      }
25
26      switch (road2_state){
27        case NORMAL_RED:
28          HAL_GPIO_WritePin(LED_RED2_GPIO_Port, LED_RED2_Pin,
     GPIO_PIN_RESET);
29          HAL_GPIO_WritePin(LED_GREEN2_GPIO_Port,
    LED_GREEN2_Pin, GPIO_PIN_SET);
30          HAL_GPIO_WritePin(LED_YELLOW2_GPIO_Port,
    LED_YELLOW2_Pin, GPIO_PIN_SET);
31          break;
32        case NORMAL_GREEN:
33          HAL_GPIO_WritePin(LED_RED2_GPIO_Port, LED_RED2_Pin,
     GPIO_PIN_SET);
34          HAL_GPIO_WritePin(LED_GREEN2_GPIO_Port,
    LED_GREEN2_Pin, GPIO_PIN_RESET);
35          HAL_GPIO_WritePin(LED_YELLOW2_GPIO_Port,
    LED_YELLOW2_Pin, GPIO_PIN_SET);
36          break;
37        case NORMAL_YELLOW:
38          HAL_GPIO_WritePin(LED_RED2_GPIO_Port, LED_RED2_Pin,
     GPIO_PIN_SET);
39          HAL_GPIO_WritePin(LED_GREEN2_GPIO_Port,
    LED_GREEN2_Pin, GPIO_PIN_SET);
40          HAL_GPIO_WritePin(LED_YELLOW2_GPIO_Port,
    LED_YELLOW2_Pin, GPIO_PIN_RESET);
41          break;
42        default:
43          break;
44      }
45    }else if (mode == MANUAL_STATE || (mode == CONFIG_STATE))
     {
46      switch (state){
47        case INIT:
48          HAL_GPIO_TogglePin(LED_RED1_GPIO_Port, LED_RED1_Pin
    );
49          HAL_GPIO_TogglePin(LED_RED2_GPIO_Port, LED_RED2_Pin
    );
50          HAL_GPIO_TogglePin(LED_YELLOW1_GPIO_Port,
    LED_YELLOW1_Pin);
51          HAL_GPIO_TogglePin(LED_YELLOW2_GPIO_Port,
    LED_YELLOW2_Pin);       HAL_GPIO_TogglePin(
```

```
  LED_RED1_GPIO_Port, LED_RED1_Pin);
52        HAL_GPIO_TogglePin(LED_GREEN1_GPIO_Port,
  LED_GREEN1_Pin);
53        HAL_GPIO_TogglePin(LED_GREEN2_GPIO_Port,
  LED_GREEN2_Pin);
54        break;
55      case BLINK_RED_GREEN:
56        clearLED();
57        HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin,
   GPIO_PIN_RESET);
58        HAL_GPIO_WritePin(LED_GREEN2_GPIO_Port,
  LED_GREEN2_Pin, GPIO_PIN_RESET);
59        break;
60      case BLINK_RED_YELLOW:
61        clearLED();
62        HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin,
   GPIO_PIN_RESET);
63        HAL_GPIO_WritePin(LED_YELLOW2_GPIO_Port,
  LED_YELLOW2_Pin, GPIO_PIN_RESET);
64        break;
65      case BLINK_GREEN_RED:
66        clearLED();
67        HAL_GPIO_WritePin(LED_RED2_GPIO_Port, LED_RED2_Pin,
   GPIO_PIN_RESET);
68        HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port,
  LED_GREEN1_Pin, GPIO_PIN_RESET);
69        break;
70      case BLINK_RED:
71        HAL_GPIO_TogglePin(LED_RED1_GPIO_Port, LED_RED1_Pin
  );
72        HAL_GPIO_TogglePin(LED_RED2_GPIO_Port, LED_RED2_Pin
  );
73        break;
74      case BLINK_YELLOW:
75        HAL_GPIO_TogglePin(LED_YELLOW1_GPIO_Port,
  LED_YELLOW1_Pin);
76        HAL_GPIO_TogglePin(LED_YELLOW2_GPIO_Port,
  LED_YELLOW2_Pin);
77        break;
78      case BLINK_GREEN:
79        HAL_GPIO_TogglePin(LED_GREEN1_GPIO_Port,
  LED_GREEN1_Pin);
80        HAL_GPIO_TogglePin(LED_GREEN2_GPIO_Port,
  LED_GREEN2_Pin);
81        break;
82      default:
83        break;
84    }
85  }
```

```
}
void displayAutoTrafficLight(){
  updateTrafficLight(0);
  switch (road1_state){
    case NORMAL_RED:
      updateLEDBuffer(1, road1_counter);
      if (isExpired(0) == 1){
        road1_state = NORMAL_GREEN;
        setTimer(0, green_time * 100);
        road1_counter = green_time;
        updateLEDBuffer(1, road1_counter);
      }
      break;
    case NORMAL_GREEN:
      updateLEDBuffer(1, road1_counter);
      if (isExpired(0) == 1){
        road1_state = NORMAL_YELLOW;
        setTimer(0, yellow_time * 100);
        road1_counter = yellow_time;
        updateLEDBuffer(1, road1_counter);
      }
      break;
    case NORMAL_YELLOW:
      updateLEDBuffer(1, road1_counter);
      if (isExpired(0) == 1){
        road1_state = NORMAL_RED;
        setTimer(0, red_time * 100);
        road1_counter = red_time;
        updateLEDBuffer(1, road1_counter);
      }
      break;
    default:
      break;
  }
  switch (road2_state){
    case NORMAL_RED:
      updateLEDBuffer(2, road2_counter);
      if (isExpired(1) == 1){
        road2_state = NORMAL_GREEN;
        setTimer(1, green_time * 100);
        road2_counter = green_time;
        updateLEDBuffer(2, road2_counter);
      }
      break;
    case NORMAL_GREEN:
      updateLEDBuffer(2, road2_counter);
      if (isExpired(1) == 1){
        road2_state = NORMAL_YELLOW;
        setTimer(1, yellow_time * 100);
```

```
135         road2_counter = yellow_time;
136         updateLEDBuffer(2, road2_counter);
137       }
138       break;
139     case NORMAL_YELLOW:
140       updateLEDBuffer(2, road2_counter);
141       if (isExpired(1) == 1){
142         road2_state = NORMAL_RED;
143         setTimer(1, red_time * 100);
144         road2_counter = red_time;
145         updateLEDBuffer(2, road2_counter);
146       }
147       break;
148     default:
149       break;
150   }
151 }
152
153 void updateCounter(){
154   if (isExpired(3) == 1){
155     setTimer(3, 100);
156     road1_counter--;
157     road2_counter--;
158
159     one_sec_tick = 1;
160   }
161   if(one_sec_tick == 1){
162     one_sec_tick = 0;
163     updateLEDBuffer(1, road1_counter);
164     updateLEDBuffer(2, road2_counter);
165   }
166 }
167 void displayTime(){
168   updateCounter();
169   if (isExpired(2) == 1){
170     setTimer(2, 10);
171     update7SEG(led_index);
172     led_index = led_index + 1;
173     if (led_index > 3){
174       led_index = 0;
175     }
176   }
177 }
178 void clearLED(){
179   HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin,
      GPIO_PIN_SET);
180   HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port, LED_GREEN1_Pin,
      GPIO_PIN_SET);
181   HAL_GPIO_WritePin(LED_YELLOW1_GPIO_Port, LED_YELLOW1_Pin,
```

```
    GPIO_PIN_SET);
182   HAL_GPIO_WritePin(LED_RED2_GPIO_Port, LED_RED2_Pin,
    GPIO_PIN_SET);
183   HAL_GPIO_WritePin(LED_GREEN2_GPIO_Port, LED_GREEN2_Pin,
    GPIO_PIN_SET);
184   HAL_GPIO_WritePin(LED_YELLOW2_GPIO_Port, LED_YELLOW2_Pin,
     GPIO_PIN_SET);
185 }
186
187 void clear7SEG(){
188   updateLEDBuffer(1, 0);
189   updateLEDBuffer(2, 0);
190 }
191
192 void displayBlinkTrafficLight(int state){
193   switch (state){
194     case BLINK_RED_GREEN:
195       if (isExpired(5)){
196         setTimer(5,500);
197         updateTrafficLight(BLINK_RED_GREEN);
198       }
199       break;
200     case BLINK_RED_YELLOW:
201       if (isExpired(5)){
202         setTimer(5,200);
203         updateTrafficLight(BLINK_RED_YELLOW);
204       }
205       break;
206     case INIT:
207       if (isExpired(5)){
208         setTimer(5,25);
209         updateTrafficLight(INIT);
210       }
211       break;
212     case BLINK_GREEN_RED:
213       if (isExpired(5)){
214         setTimer(5,500);
215         updateTrafficLight(BLINK_GREEN_RED);
216       }
217       break;
218     case BLINK_RED:
219       if (isExpired(5)){
220         setTimer(5,50);
221         updateTrafficLight(BLINK_RED);
222       }
223       break;
224     case BLINK_GREEN:
225       if (isExpired(5)){
226         setTimer(5,50);
```

```
227         updateTrafficLight(BLINK_GREEN);
228       }
229       break;
230     case BLINK_YELLOW:
231       if (isExpired(5)){
232         setTimer(5,50);
233         updateTrafficLight(BLINK_YELLOW);
234       }
235       break;
236     default:
237       break;
238   }
239 }
```

Program 19: display_led.c

1. **void updateTrafficLight(int state)** is used to update the traffic light system based on the current mode and type-in state.
2. **void displayAutoTrafficLight()** is used to display the mornal traffic light system (RED -> Green -> Amber -> Red)
3. **void displayBlinkTrafficLight()** is mainly used to display the LEDs' blink state, but i have put in some others states for further development
4. **void displayTime()** is used to display countdown numbers on 7-segment LEDs (using led scanning)

5.4.6 state_machine.h/ state_machine.c

```
1  #ifndef INC_STATE_MACHINE_H_
2 #define INC_STATE_MACHINE_H_
3 #include "global.h"
4
5 void fsm_run();
6
7 #endif /* INC_STATE_MACHINE_H_ */
```

Program 20: state_machine.h

In the **state_machine** source code, there is only one function called **void fsm_run()** to run the state machine including all states in the system.

```
1  #include "global.h"
2  int temp_duration;
3  void fsm_run(){
4  //  updateCounter();
5    switch (mode){
6      case INIT:
7        clearLED();
8        mode = AUTO_STATE;
9        setTimer(0, red_time * 100);
10       setTimer(1, green_time * 100);
11       setTimer(2, 10);
12       setTimer(3, 100);
```

```
13      setTimer(5,50);
14      road1_counter = red_time;
15      road2_counter = green_time;
16      break;
```

Program 21: state_machine.c - INIT state

This is **INIT_STATE**: in this we set all needed timer, set the next mode and declare some inital values.

```
1     case AUTO_STATE:
2       displayAutoTrafficLight(0);
3       displayTime();
4       if(isButtonPressed(0)){
5         mode = MANUAL_STATE;
6         manual_blink_mode = BLINK_RED_GREEN;
7         updateLEDBuffer(2, 2);
8         setTimer(2, 25);
9         setTimer(5, 50);
10        clear7SEG();
11        clearLED();
12        break;
13      }
14      break
15      break;
```

Program 22: state_machine.c - AUTO_STATE

This is **AUTO_STATE**: This is the normal 2 ways traffict light system with the duration of red, yellow and green lights are 9s, 6s and 2s, respectively.

```
1     case MANUAL_STATE:
2       switch (manual_blink_mode){
3         case BLINK_RED_GREEN:
4           if (manual_blink_mode != BLINK_RED_GREEN){
5             manual_blink_mode = BLINK_RED_GREEN;
6           }
7           if(isButtonPressed(1)){
8             manual_blink_mode = BLINK_RED_YELLOW;
9             updateLEDBuffer(2,2);
10            break;
11          }
12          if(isExpired(2)){
13            setTimer(2,10);
14            update7SEG(led_index++);
15            if (led_index > 3){
16              led_index = 0;
17            }
18          }
19          break;
20        case BLINK_RED_YELLOW:
21          if (manual_blink_mode != BLINK_RED_YELLOW){
22            manual_blink_mode = BLINK_RED_YELLOW;
```

```
        }
        if(isExpired(5)){
          if(manual_blink_mode != BLINK_GREEN_RED){
            manual_blink_mode = BLINK_GREEN_RED;
            break;
          }
          updateLEDBuffer(2,2);
        }
        if(isExpired(2)){
          setTimer(2,10);
          update7SEG(led_index++);
          if (led_index > 3){
            led_index = 0;
          }
        }
        break;
      case BLINK_GREEN_RED:
        if (manual_blink_mode != BLINK_GREEN_RED){
          manual_blink_mode = BLINK_GREEN_RED;
        }
        if(isButtonPressed(1)){
          if(manual_blink_mode != BLINK_RED_GREEN){
            manual_blink_mode = BLINK_RED_GREEN;
          }
          updateLEDBuffer(2,2);
          break;
        }
        if(isExpired(2)){
          setTimer(2,10);
          update7SEG(led_index++);
          if (led_index > 3){
            led_index = 0;
          }
        }
        break;

    }
    if (isButtonPressed(0)){
      mode = CONFIG_STATE;
      config_mode = CONFIG_RED;
      manual_blink_mode = BLINK_RED;
      setTimer(2, 10);
      setTimer(5, 50);
      clearLED();
      temp_duration = red_time;
      updateLEDBuffer(1, temp_duration);
      updateLEDBuffer(2, 3);
      break;
    }
```

```
72      displayBlinkTrafficLight(manual_blink_mode);
73
74      if(isExpired(2)){
75        setTimer(2,10);
76        update7SEG(led_index++);
77        if (led_index > 3){
78          led_index = 0;
79        }
80      }
81      break;
```

Program 23: state_machine.c - MANUAL_STATE

This is **MANUAL_STATE**: when you click the first button, it will change to manual state and turn the led red on 1 way, the other wat turn green. If you push the second button it will turn opposite (with 2 second delay of yellow and red)

```
1     case CONFIG_STATE:{
2       switch (config_mode){
3         case CONFIG_RED:
4           if (manual_blink_mode != BLINK_RED){
5             manual_blink_mode = BLINK_RED;
6             clearLED();
7           }
8           displayBlinkTrafficLight(manual_blink_mode);
9           if (isButtonPressed(1)){
10            temp_duration++;
11            if (temp_duration > 99){
12              temp_duration = 0;
13            }
14            updateLEDBuffer(1, temp_duration);
15          }
16          if (isExpired(2)){
17            setTimer(2, 10);
18            update7SEG(led_index++);
19            if (led_index > 3){
20              led_index = 0;
21            }
22
23          }
24          if (isButtonPressed(2)){
25            red_time = temp_duration;
26            break;
27          }
28          if (isButtonPressed(0)){
29            config_mode = CONFIG_GREEN;
30            manual_blink_mode = BLINK_GREEN;
31            temp_duration = green_time;
32            setTimer(2, 10);
33            setTimer(5, 50);
34            clearLED();
```

```
            updateLEDBuffer(1, temp_duration);
            updateLEDBuffer(2, 3);
            break;
          }
          break;
        case CONFIG_GREEN:
          if (manual_blink_mode != BLINK_GREEN){
            manual_blink_mode = BLINK_GREEN;
            clearLED();
          }
          displayBlinkTrafficLight(manual_blink_mode);
          if (isButtonPressed(1)){
            temp_duration++;
            if (temp_duration > 99){
              temp_duration = 0;
            }
            updateLEDBuffer(1, temp_duration);
          }
          if (isExpired(2)){
            setTimer(2, 10);
            update7SEG(led_index++);
            if (led_index > 3){
              led_index = 0;
            }

          }
          if (isButtonPressed(2)){
            green_time = temp_duration;
            updateLEDBuffer(1, temp_duration);
            updateLEDBuffer(2, 3);
            break;
          }
          if (isButtonPressed(0)){
            config_mode = CONFIG_YELLOW;
            temp_duration = yellow_time;
            manual_blink_mode = BLINK_YELLOW;
            setTimer(2, 10);
            setTimer(5, 50);
            clearLED();
            updateLEDBuffer(1, temp_duration);
            break;
          }
          break;
        case CONFIG_YELLOW:
          if (manual_blink_mode != BLINK_YELLOW){
            manual_blink_mode = BLINK_YELLOW;
            clearLED();
          }
          displayBlinkTrafficLight(manual_blink_mode);
```

```
            if (isButtonPressed(1)){
              temp_duration++;
              if (temp_duration > 99){
                temp_duration = 0;
              }
              updateLEDBuffer(1, temp_duration);
            }
            if (isExpired(2)){
              setTimer(2, 10);
              update7SEG(led_index++);
              if (led_index > 3){
                led_index = 0;
              }

            }
            if (isButtonPressed(2)){
              yellow_time = temp_duration;
              updateLEDBuffer(1, temp_duration);
              break;

            }
            if (isButtonPressed(0)){
              mode = AUTO_STATE;
              road1_counter = red_time;
              road2_counter = green_time;
              road1_state = NORMAL_RED;
              road2_state = NORMAL_GREEN;
              setTimer(0, red_time * 100);
              setTimer(1, green_time * 100);
              updateTrafficLight(0);
              break;
            }
            break;
          default:
            break;
        }
        break;
      }
      default:
        break;

  }
}
```

Program 24: state_machine.c - MANUAL_STATE

This block is **CONFIG_STATE**: From **MANUAL_STATE** if you want to change to config state, you have to push the first button. In this state:

- **First button**: to choose which color being modified
- **Second button**: is used to increase the duration.
- **Third button**: is used to save the configuration.

6 Repository and Document Information

Root Repository (All labs:*HCMUT_STM32_2310190_LAB*
Lab 3 Source code: *2310190_LAB3*
The source code for all exercises in Lab 2 is organized and maintained within the corresponding sub-folders of the main repository

Author: *Trương Thiên Ân*
Last update: *November 6th, 2025*